コンピュータ科学科
知能システム系

卒業論文

個別楽器音に基づく
楽曲間類似度表現学習における音源分離の活用法

2024年2月

102030049 今村 剛大

目 次

第 1 章　序論 **1**

1.1　はじめに 1

1.2　本研究の目的 3

1.3　本論文の構成 4

第 2 章　関連研究 **5**

2.1　はじめに 5

2.2　各楽器音に着目した楽器間類似度学習 5

2.3　個別楽器音に基づいた楽曲間類似度のための分離表現学習 7

2.4　U-Net を用いた音源分離手法 11

2.5　従来手法の問題点 12

2.6　まとめ 13

第 3 章　提案手法 **15**

3.1　はじめに 15

3.2　提案手法 1：Disent-Reconst 15

3.2.1　事前学習 15

3.2.2　ネットワーク構造 17

3.2.3　入力データ 18

3.2.4　同時学習における損失関数 19

3.3 提案手法 2：Serial-FT 20
3.3.1 ネットワーク構造 20
3.3.2 音源分離モデルの学習 21
3.3.3 特徴量抽出器の学習 22
3.3.4 fine-tuning 22
3.4 まとめ 23

第 4 章 実験的評価 **25**
4.1 はじめに 25
4.2 実験条件 25
4.2.1 データセット 25
4.2.2 提案手法 1 に対する実験 26
4.2.3 提案手法 2 に対する実験 26
4.3 評価方法 26
4.3.1 テスト楽曲に対する特徴表現の精度評価 26
4.3.2 テスト用擬似楽曲に対する特徴表現の精度評価 27
4.3.3 主観評価 28
4.4 結果 29
4.4.1 提案手法 1 に対する評価結果 29
4.4.2 提案手法 2 に対する評価結果 30
4.5 まとめ 32

第 5 章 結論 **39**
5.1 本論文のまとめ 39
5.2 今後の課題 40

謝辞 **43**

参考文献 **45**

第1章　序論

1.1　はじめに

音楽市場は年々拡大し続けているが，これは近年のインターネットの普及により，媒体が CD などの物理的形式からストリーミングなどのデジタル形式へ移行しつつあることが大きく影響していると考えられる．2022 年の音楽市場全体の売上は 262 億アメリカドルで，前年から 22 億増であり，8 年連続で上昇している．また，そのうち音楽ストリーミングサービスでの売上は 184 億アメリカドルで，過去最大を更新しており，市場全体の約 67%を占めている [1]．中でも世界的なシェアを集めているサブスクリプションサービス Spotify では，2023 年現在アクティブユーザ数が 5 億 5100 万人であり [2]，音楽ストリーミング市場全体で共有されている楽曲数は約 1 億 5800 万曲であると言われている [3]．音楽市場は音楽ストリーミングサービスの更なる成長に伴い，今後も拡大を続けると予想される．

このような状況から，サブスクリプションサービスのユーザは大量の曲の中から好みの曲を見つける必要に迫られるが，全ての曲を聞いて探すことは不可能である．現に，この楽曲 1 億 5800 万曲の中で 2022 年度に 1 回も再生されていない曲は約 3800 万曲も存在するというデータがある [3]．このことから，曲とユーザのマッチングを促すことができるようなシステムは，ユーザ・アーティスト，さらには市場にとっても有益なものとなるであろう．

ユーザが幅広い音楽からより効率よく好みの曲を見つけることを可能にするために有効なのは，楽曲推薦システムなどの音楽情報検索 (Music Information Retrieval: MIR)

技術である．MIR 技術の中で最も一般的に用いられているものは，テキストによるメタデータ検索や協調フィルタリングという手法 [4] であるが，メタデータ検索には表現力に限界があるという問題，協調フィルタリングにはコールドスタート問題 [4] や，人気の楽曲が推薦されやすいという問題 [4] がある．そのため，楽曲自体の特徴を推薦，検索に用いるコンテンツベースの手法の実現が望まれている．コンテンツベース手法では，楽曲固有の特徴量を抽出し，特徴量同士の距離を計算したり，分類アルゴリズムに入力することにより，コンテンツ同士の類似度を算出し，それに基づき推薦，検索を行う [5]. 近年では，深層学習によってコンテンツ特徴を高い精度で計算する手法も登場しており，その性能の向上が見られる．例えば，ニューラルネットワークを用いて，ジャンル推定モデルの中間層から特徴量を出す手法 [6] や，ジャンルやメタデータに関する教師ラベルを用いて楽曲の特徴量やラベルを推定する手法 [7] などでは，ニューラルネットワークを用いて，楽曲から高い精度で特徴量を抽出することに成功している．しかし一般的にこれらの手法は楽曲全体に対する包括的な特徴量を出力しているため，個人の嗜好を反映した推薦システムの構築には適していないと言える．またジャンルやラベルのついた教師データを得るには人的コストを要してしまうという問題点も存在する．

そこで，単一の楽曲から複数の視点に基づく特徴量を抽出する分離表現学習を試みた研究が多数行われている．音楽においては，音色とピッチの分離表現を獲得する研究 [8][9] において単一の楽曲から複数の特徴量を出力することを達成している．また，個別楽器音の特徴に基づいた楽曲推薦手法についての研究 [10][11] では，色々な楽器音で構成されている楽曲の中から特定の楽器音の特徴量を抽出することで，例えば楽曲の中のドラム音が似ている曲を検索するなどといったより柔軟な検索を行うことを可能にしている．さらに，楽曲間類似度学習を，同じ楽曲内から切り取ったサンプル同士は似ているという仮定に基づいて，トリプレットロスを用いた距離学習 [12] によって行うことによってラベルなしでの学習を可能にしており，人的コストの抑制に対し

ても有効な手法である.

このような個別楽器音特徴に基づいた楽曲間類似度計算手法は，2 つに大別される．1 つ目は，個別に学習された音源分離モデルと各楽器音特徴量抽出器を直列に接続する手法である（Serial）．ここで，各楽器音特徴抽出器は，分離楽器音を用いて，同一曲内の分離楽器音からは類似した特徴量が得られるように学習する．この手法では分離モデルは独立に学習されるため，必ずしも楽曲類似度計算に適した分離楽器音が得られるとは限らない．これに対して，2 つ目は音源分離を適用せず，楽曲から単一の特徴量抽出器を用いて，各楽器音特徴を捉えた楽曲特徴量を抽出する手法である（Disent）．この手法では，各楽器音に対して異なる部分空間を持つ特徴量空間を学習する必要があるが，そのような学習は容易ではなく，一部の楽器に対しては十分な楽曲間類似度計算精度が得られていないという課題がある．

1.2 本研究の目的

本論文では，個別楽器音に基づいた楽曲間類似度計算手法である Disent と Serial に対して，音源分離を活用した改良を加えて性能の改善を試みる．Disent に対してはデータ拡張（Data Augmentation: DA）と楽器音の再構成（Reconstruction: Reconst）を同時学習する手法（図 1.1 左）を，Serial に対しては音源分離モデルに事前学習を行い，さらに End-to-End な fine-tuning（FT）を行う手法（図 1.1 右）を提案する．提案手法によって，Disent においては各部分空間に個別楽器音特徴がより多く反映することを期待し，Serial においてはそれぞれの特徴量抽出器から各楽器音特徴をより反映した特徴量が出力されることを期待する．楽曲 MIDI 音源とその楽曲中の楽器音 MIDI 音源を含むデータセット slakh [13] を用いた実験的評価により，1）Disent に対する DA と Reconst の同時学習が有効であること，2）Serial に対する音源分離モデルの事前学習と FT が有効であることを明らかにする．

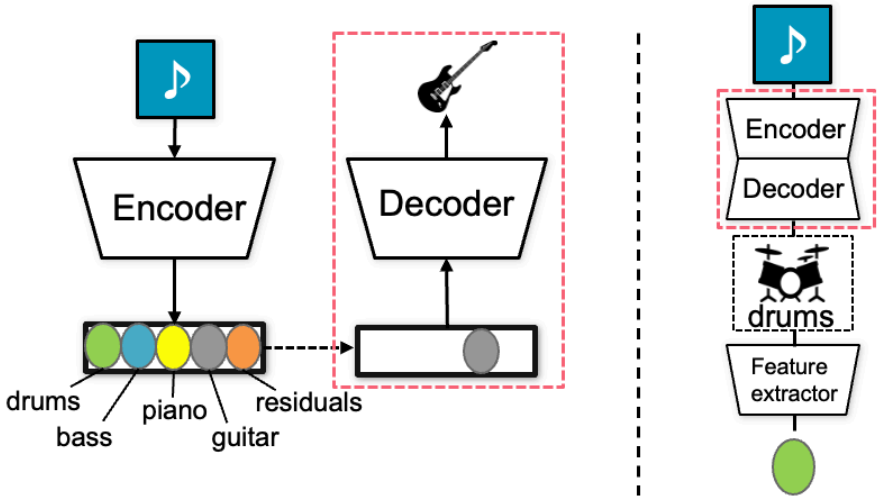


図 1.1: 本研究の概要．Disent（左），Serial（右）それぞれに対してピンクの点線枠部分を追加・改良する．

1.3　本論文の構成

本論文の構成は以下の通りである．第 2 章にて，個別楽器音特徴に基づいた楽曲間類似度計算手法である Disent と Serial，また，Serial で用いられる音源分離手法について述べる．第 3 章にて，提案手法として，Disent と Serial に対する改良点について述べる．第 4 章にて，各提案手法の有効性を示すための実験的評価方法とその評価指標，そして評価結果について述べる．最後に第 5 章にて，本論文の結論と今後の課題を述べる．

第2章　関連研究

2.1　はじめに

本章では，各楽器音特徴に基づいた楽曲間類似度を計算する先行研究について述べ，その後音源分離手法の先行研究について述べる．

2.2　各楽器音に着目した楽器間類似度学習

一般的に楽曲は様々な楽器音が混ざり合って構成されているため，楽曲中の各個別楽器音特徴に基づいた推薦 [10] は，ユーザの嗜好をより詳細に反映した楽曲推薦を可能にできるため，その実現が期待される．そこでこの研究では楽曲中の各個別楽器音特徴量を計算する手法について提案している．

この手法 [10] の概要を図 2.1 に示す．この手法では，トリプレット損失を用いた距離学習によって特徴量抽出器を学習する．入力として用いるのは各個別楽器音（drums 音，bass 音など）であり，短時間のセグメントに分割し，メルスペクトログラムに変換して特徴量抽出器に入力する．特徴量抽出器は主に CNN または FCN で構成されており，入力楽器音の特徴を表す 128 次元ベクトルを出力する．学習に用いるトリプレット損失は，基準となるデータであるアンカー $\mathbf{x}_i^{(a)}$，アンカーと類似したデータと定義されるポジティブ $\mathbf{x}_i^{(p)}$，そしてアンカーと類似していないと定義されるネガティブ $\mathbf{x}_i^{(n)}$ の 3 つのデータを使用し，距離関数を $d(\cdot)$ で表すと，式 2.1 で定式化できる．この研究では同じ楽曲から取り出されたセグメントは似ているという仮定のもとラベルなしで学

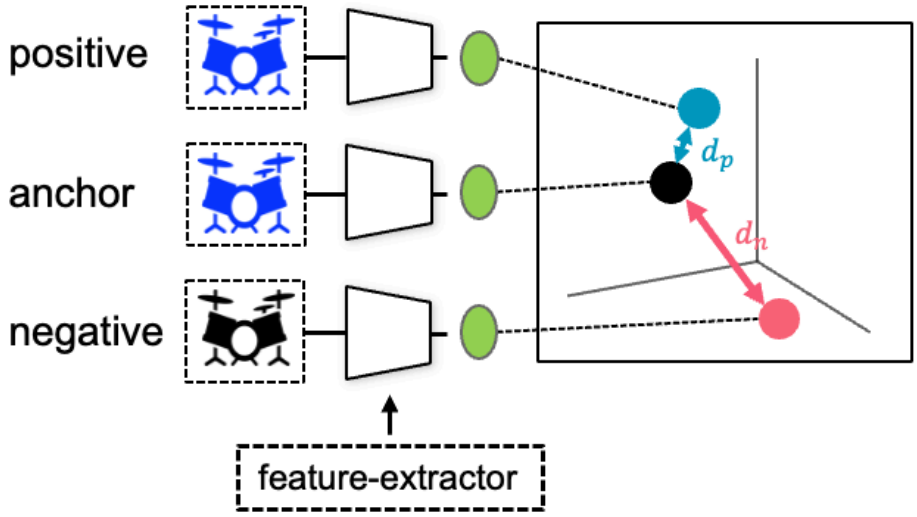


図 2.1: 各楽器音に着目した楽器間類似度学習 [10] の概要．この例では clean な個別楽器音を用いた場合の drums における学習の例を示している．学習ではアンカーの特徴量とポジティブの特徴量間の距離 d_p が短く，アンカーの特徴量とネガティブの特徴量間の距離 d_n が長くなるように学習する．drums の画像は同じ色は同じ楽曲のセグメントであることを示す．

習するため，アンカーのセグメントに対し，ポジティブは同じ楽曲から取り出されたセグメントを用い，ネガティブは異なる楽曲から取り出されたセグメントを使用する．

$$\mathcal{L}_{\text{triplet}}(\mathbf{x}_i^{(a)}, \mathbf{x}_i^{(p)}, \mathbf{x}_i^{(n)}) = \max\{0, d(\mathbf{x}_i^{(a)}, \mathbf{x}_i^{(p)}) - d(\mathbf{x}_i^{(a)}, \mathbf{x}_i^{(n)}) + \delta\} \tag{2.1}$$

なお，δ はアンカー・ポジティブ間とアンカー・ネガティブ間の最小距離を定義するマージンである．

入力の個別楽器音源は，clean な音源が手に入る場合はそのまま clean 音源を使用する（各個別楽器の clean な音源を用いたトリプレット学習手法を以下 Clean と呼称する）．ただ楽曲中の clean な個別楽器音源は一般的には公開されていない場合が多く，clean な音源が手に入らない場合，音源分離手法等を使用して得られる推定分離楽器音

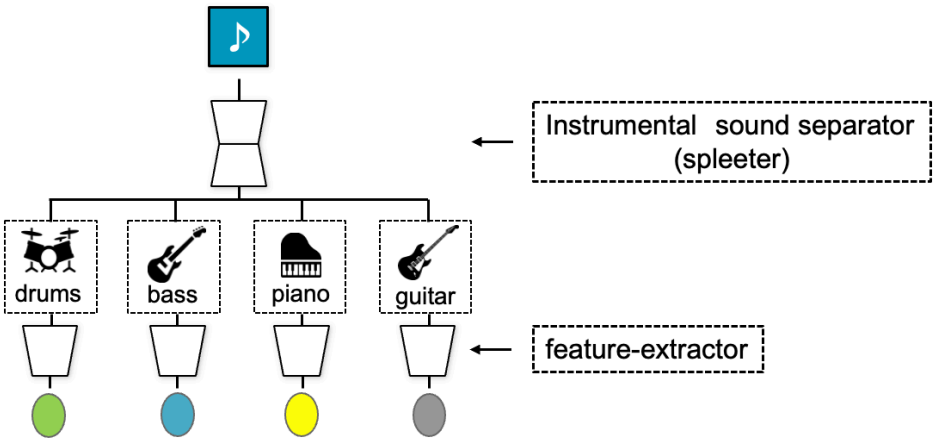


図 2.2: 音源分離モデルを使用する際（Serial）の先行研究 [10] のネットワークの概要図．楽曲音源を音源分離モデル（spleeter [14]）で各個別楽器音に分離し，それぞれを各特徴量抽出器に入力して楽器音特徴量を得る．

を使った特徴量計算を余儀なくされる．その場合，音源分離モデルと特徴量抽出器を直列に繋げたモデル （Serial）（図 2.2）を使用することになるが，先行研究 [10] では音源分離モデルに python ライブラリとして公開されている spleeter [14] の学習済みネットワークを使用していた．しかし，音源分離モデルによる音質劣化や欠損・余剰音などが起因して，clean な楽器音を用いた場合と比較して，必ずしも楽曲類似度計算に適した分離楽器音が得られるとは限らなくなってしまう問題がある．現に，先行研究 [10] では分離音を用いた場合に楽曲間類似度計算精度が低下することが確認されている．

2.3 個別楽器音に基づいた楽曲間類似度のための分離表現学習

分離音を用いることによる類似度計算精度低下の影響を回避するため, 個別楽器音に基づいた楽曲間類似度のための分離表現学習 [11] という研究では, 単一の特徴量抽出器に楽

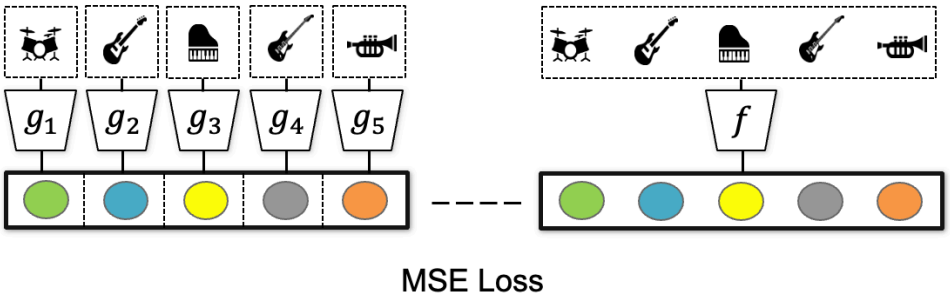


図 2.3: 個別楽器音に基づいた楽曲間類似度のための分離表現学習 [11] における事前学習．この図は楽曲そのものを表す mix 音を入力として使用した事前学習の概要図である．

曲音源そのものを入力して, 各楽器音の特徴量を部分空間に分離して（Disentanglement: Disent）出力することを目指している．部分空間ごとに分離した表現を出力することで，楽曲音源特徴同士の相違からどの楽器音は異なっていて，どの楽器音が似ているのかを表現することができる．具体的には，出力される特徴量ベクトル 640 次元ベクトル中 128 次元ずつが各個別楽器音特徴量となるように学習する．

学習はまず，事前学習として 2.2 節で述べた，Clean の出力学習埋め込み特徴量を教師データとして，Disent の特徴量抽出器の出力と教師データとで平均 2 乗誤差（Mean Squared Error: MSE）を計算し学習を行う． 図 2.3，2.4 は，事前学習の概要図である．$g_1(\cdot)$, $g_2(\cdot)$, $g_3(\cdot)$, $g_4(\cdot)$, $g_5(\cdot)$ はそれぞれ clean な楽器音入力の楽器音特徴量抽出器（2.2 節で前述）を表しており，$f(\cdot)$ は Disent の学習対象特徴量抽出器を表している．clean な楽器音入力の楽器音特徴量抽出器の出力を結合したものを教師データとし，Disent の学習対象特徴量抽出器の出力との MSE 損失を計算する．学習は 2 パターンの入力に対して行う．1 パターン目は楽曲音源そのものである mix 音源を入力とする場合（図 2.3）で，2 パターン目は各楽器音源を入力とする場合（図 2.4）である．2 パターン目の場合は Disent の入力が各楽器音源なので，対象楽器以外の Clean の特徴量抽出器の入力は無音になる．その場合は，無音を入力した時の特徴量抽出器の出力

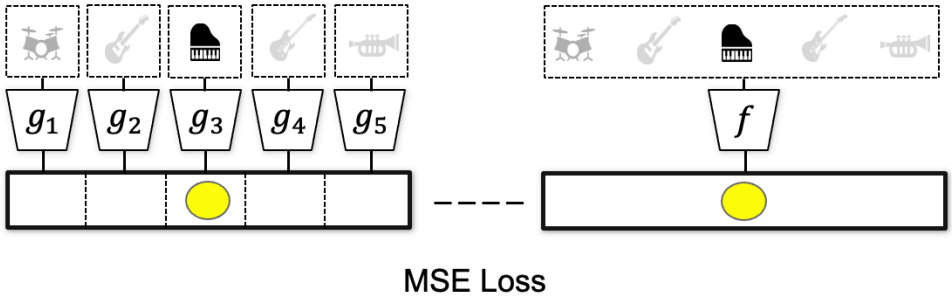


図 2.4: 個別楽器音に基づいた楽曲間類似度のための分離表現学習 [11] における事前学習．この図は各楽器音源を入力として使用した事前学習の概要図である．

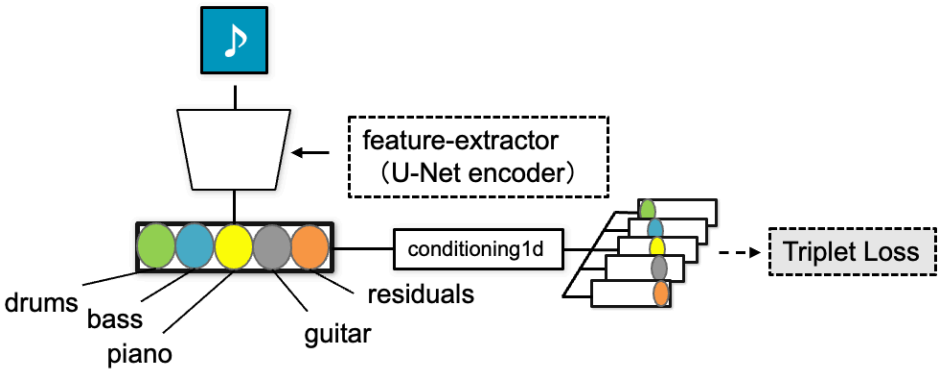


図 2.5: 個別楽器音に基づいた楽曲間類似度のための分離表現学習 [11] の概要図．

ベクトルを用いることはせず，出力ベクトルが零ベクトルになると仮定して，零ベクトルを教師データとして用いる．

その後，事前学習済み特徴量抽出器に対してトリプレット損失（式 2.1）を用いた学習を行う．トリプレット損失を用いた学習の概要が図 2.5 である．この学習法は，画像分野において，例えば靴の色や，カテゴリなどの局所的な特徴の類似度を部分空間に分けて表現するという学習法 [15] を，音楽において各楽器音の類似度で適用させたものである．同じ学習法を音楽に適応させた先行研究 [16] は存在するが，教師ラベルを用いた類似度学習として実現しているため，教師データにラベルを付与する際に，通

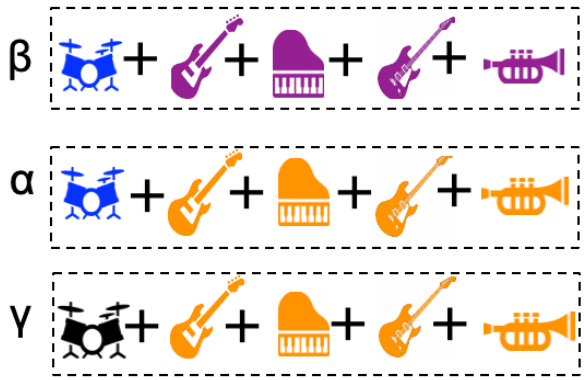


図 2.6: トリプレット学習で用いられた擬似楽曲．同じ色は同じ楽曲の楽器音セグメントであることを示している．

常，人的コストを必要とする．それに対し，図 2.5 で示されている学習法はラベルなしでトリプレット損失を用いた学習を実現している．これにより，通常，教師データにラベルを付与する際に必要となる人的コストが不要となる．一方で，通常の楽曲を用いたトリプレット学習では，楽曲単位でラベルが付与されてしまうため，各楽器音に対するラベルが同一になってしまい，注目する楽器音に対して学習を行う場合であっても，注目する楽器音の特徴とそれ以外の楽器音の特徴を区別して扱うことができなくなってしまうため，十分な精度で各楽器音に対する部分空間を学習するのは困難である．そこで，先行研究では異なる楽曲の楽器音を合成して擬似楽曲を作成して学習に用いている．作成する擬似楽曲の概要図は図 2.6 である．図 2.6 の擬似楽曲 α，β，γ において，楽曲 α と楽曲 β は drums においては同一の楽曲のセグメントであるが，そのほかの楽器音においては異なる楽曲のセグメントである．また楽曲 α と楽曲 γ は drums においては異なる楽曲のセグメントであるが，そのほかの楽器音においては同一の楽曲のセグメントである．この 3 つの楽曲を，トリプレット損失（式 2.1）を計算する際，アンカー $\mathbf{x}_i^{(a)}$ に楽曲 α，ポジティブ $\mathbf{x}_i^{(p)}$ に楽曲 β，ネガティブ $\mathbf{x}_i^{(n)}$ に楽曲 γ を用いることで drums 特徴のみに着目したトリプレット学習を行うことができる．

特徴量抽出器 (図 2.3，2.4，2.5 では $f(\cdot)$ で表されている) は，畳み込み層を 10 層重

ねた構造をしており，2.4 節で述べている U-Net [17] のエンコーダと同じ構造である．U-Net エンコーダの出力はチャンネル方向、周波数方向、時間方向が存在する 3 次元行列なので、時間方向に平均をとったのち、平坦化を行ってベクトルに変換した後、全結合層を使用して 640 次元ベクトルに整形する。

またネットワークの出力は 640 次元でそのうち 128 次元ずつが各楽器音特徴を表すようにトリプレット学習するために，例えば drums について損失を計算する場合は drums を担う 128 次元以外に 0 をかけてマスクして drums 特徴次元のみ 0 以外の値をもつような状態で損失を計算する．basic という手法では図 2.6 でいう drums 部分で学習を行い，additional という手法では図 2.6 でいう drums 以外の楽器から一つをランダムで選択して学習をする．先行研究 [11] において，basic と additional を同時に学習させることで性能が向上することが確認されている。またさらに部分空間に分けた学習を行うために，事前学習で用いた学習法のうち楽曲そのものを入力する方法 (図 2.4) を補助損失として取り入れている．

しかし各楽器音に対して異なる部分空間を持つ特徴量空間を学習することは容易ではなく，一部の楽器に対しては十分な楽曲間類似度計算精度が得られていないというのが現状である．

2.4 U-Net を用いた音源分離手法

U-Net [17] は医療画像のセマンティックセグメンテーションで用いられたモデルである．U-Net の構造はオートエンコーダと似ていて，入力をエンコーダとデコーダで処理して，入力と同じサイズのデータが出力される．しかしオートエンコーダと異なる点として，エンコーダ・デコーダの各層の間にスキップ接続が存在する．このスキップ接続でエンコーダの出力を次のエンコーダに渡すだけなく，同じ階層のデコーダにも渡すことができる．このスキップ接続によって，デコーダに高周波の情報を与える

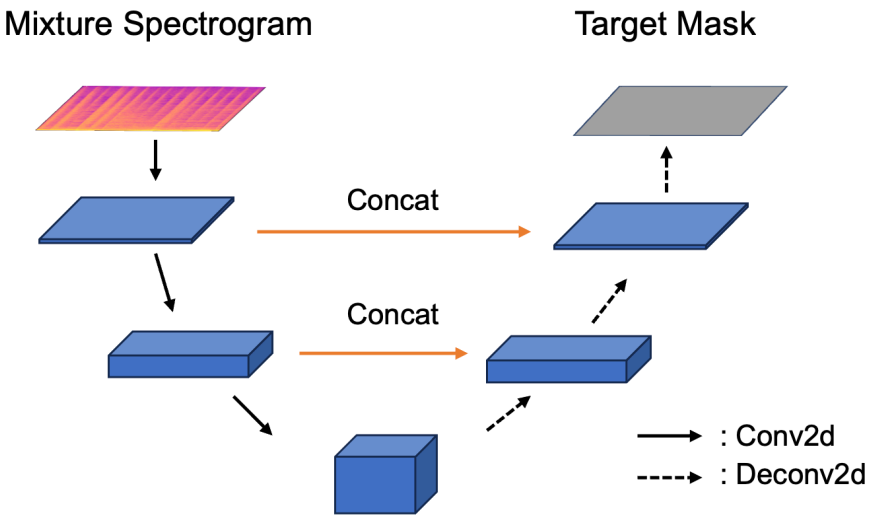


図 2.7: 音源分離で利用された U-Net の概要図．この図では 3 層のエンコーダ・デコーダだが，実際は 6 層ある．

ことができるため，出力画像を高解像度化することができる [18].

この U-Net を楽曲音源分離に応用した研究が多数行われている [18][14]．音源分離において用いられた U-Net モデルの概要図が図 2.7 である．音源分離における U-Net の入力は混合音スペクトログラムで出力は抽出したい分離音のマスクである。このマスクと入力混合音スペクトログラムとをかけ合わせることで、目的の分離音スペクトログラムを得ることができる。

2.5　従来手法の問題点

従来手法の問題点として，各楽器音をそれぞれ特徴量抽出器に入力するアプローチでは，楽曲中の楽器音の clean 音源を入手できない場合，音源分離モデルと特徴量抽出器を直列（Serial）に接続する必要があるが，音源分離モデルは独立に学習されるため，

必ずしも楽曲類似度計算に適した分離楽器音が得られるとは限らないという問題点がある．さらに楽曲音源を単一の特徴量抽出器に入力し，各楽器音特徴量を異なる部分空間に分離 （Disent）して出力するアプローチでは，一部の楽器に対しては十分な楽曲間類似度計算精度が得られていないという点が課題である．

2.6 まとめ

第 2 章では，本研究に関連する先行研究について述べた．まず，2.2 節では，楽曲中の各個別楽器音特徴量に基づく楽曲間類似度を出力する手法における，音源分離モデルと特徴量抽出器を直列（Serial）に接続する手法について述べ，2.3 節では，同じく各個別楽器音特徴量に基づく楽曲間類似度を出力する手法における，単一の特徴量抽出器に楽曲音源そのものを入力して，各楽器音の特徴量を部分空間に分離して（Disentanglement: Disent）出力する手法について述べ，またそれぞれの手法における問題点についても触れた．さらに 2.4 節では，本論文の提案手法に関わる音源分離手法について述べた．

第3章　提案手法

3.1　はじめに

提案する手法は Serial[10]，Disent[11] に対して音源分離を活用して精度向上を目指したものである．

まず Disent に対しては，U-Net [17] エンコーダ部分と同じ構造である特徴量抽出器に，デコーダを追加して楽器音の再構成を同時学習できるように改良する．また Serial に対しては Spleeter [14] の学習済みモデルを使用していた音源分離モデルを，本タスクに適応できるように学習可能にして，音源分離モデルに対する事前学習と End-to-End な fine-tuning を導入する．

3.2　提案手法1：Disent-Reconst

まず，Disent の精度向上のために，分離音の再構成 （Reconstruction: Reconst）を同時学習させる手法を提案する．

3.2.1　事前学習

先行研究 [11] と同様まずネットワークを事前学習させる．先行研究 [11] では入力として楽曲音源そのものである mix 音源と個別楽器音源の 2 パターンを用いていたが，提案手法では楽器音源の全組み合わせパターンを用いる．事前学習の概要は図 3.1 であ

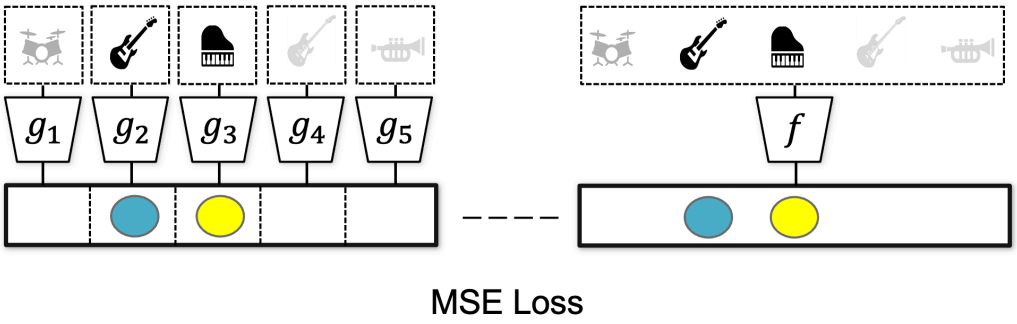


図 3.1: 音源分離を追加した個別楽器音に基づいた楽曲間類似度のための分離表現学習の事前学習の概要図．この図は例として bass と piano が有音でその他の楽器音は無音である入力を用いた場合の学習を示している．

る．本論文では，楽曲の drums，bass，piano，guitar，residuals の 5 種類を用いるため，全組み合わせの数は 2^5 通りとなり，さらにその組み合わせ数から無音を除いた計 31 通りを入力として用いる．

i 番目のデータ $\mathbf{x}_i$ に対する事前学習の流れを述べる．まず各個別楽器音源に対して有音にするか否かをランダムに選択し，足し合わせることで，$\mathbf{x}_i$ を作成する．そして，2.3 節でも述べた手法と同様に，有音の楽器次元に対しては Clean の出力学習埋め込み特徴量の値を，無音の楽器次元に対しては零ベクトルを当てはめた 640 次元ベクトル $\mathbf{y}_i$ を教師データとして用いて，Disent の特徴量抽出器の出力と教師データとで平均 2 乗誤差 （Mean Squared Error: MSE）を計算し学習を行う．なお，教師データは 640 次元に結合後，L2 ノルムの正規化を行う．ネットワークを関数 $f(\cdot)$ で表すと，損失関数は以下で定式化できる．

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(f(\mathbf{x}_i) - \frac{\mathbf{y}_i}{\|\mathbf{y}_i\|_2})^2 \tag{3.1}$$

なお n は Disent の出力特徴量ベクトル長である．

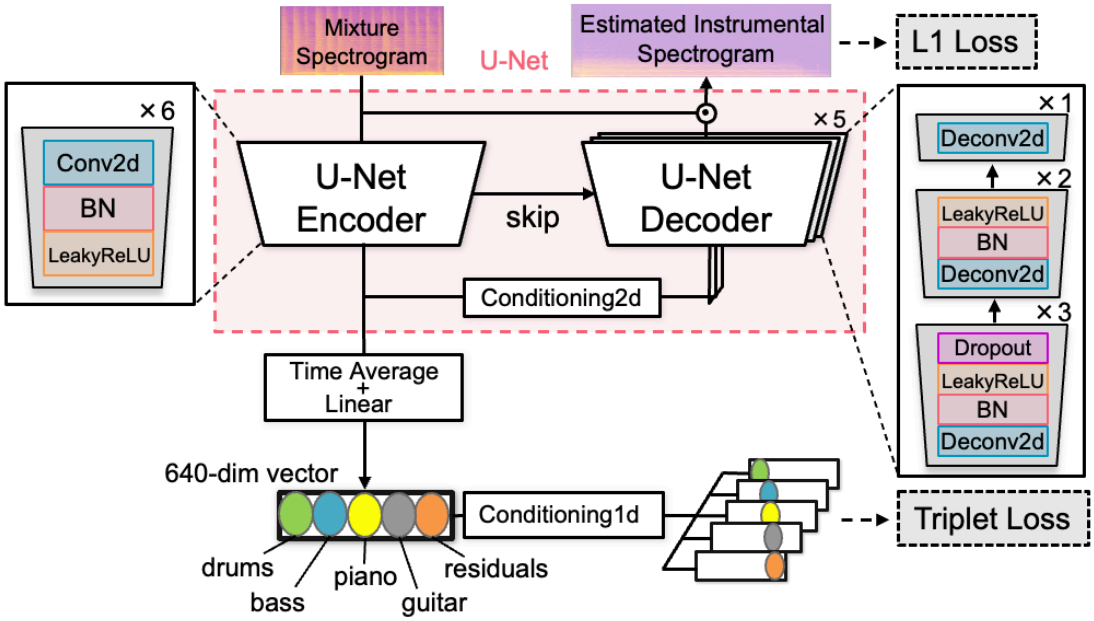


図 3.2: 提案手法 1：Disent-Reconst の概要図.

3.2.2 ネットワーク構造

提案手法のネットワークの概要図は図 3.2 である．Disent のネットワーク部分は，U-Net [17] のエンコーダ部分のみで，図 3.2 の U-Net の左半分のみである．そこで提案手法では，エンコーダの出力が各個別楽器音特徴で分離されて出力される働きを促進できるようにするため，その U-Net エンコーダに対してデコーダを新たに追加する．デコーダの出力は，音源分離における U-Net [18] と同じく，対象個別楽器音の分離音マスクである．また，エンコーダの出力と同様，デコーダの入力においても各楽器音特徴が分離されるようにするために，エンコーダ出力の 640 次元ベクトルに対して行っている条件付け (図 3.2 の Conditioning1d) と同様の条件付けをデコーダの入力にも行う (図 3.2 の Conditioning2d)．640 次元ベクトルに対して行う条件付けとデコーダの入力に対して行う条件付けの概要図が図 3.3 である．エンコーダの出力はチャンネル方向，時間方向，周波数方向の存在する 3 次元行列であるが，特徴埋め込みベクトルは

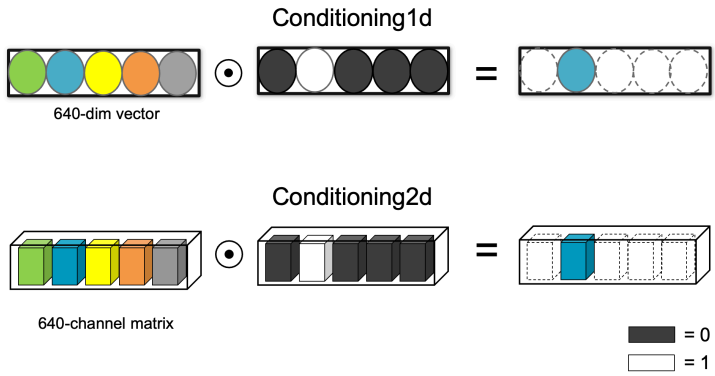


図 3.3: 図 3.2 における Conditioning1d と Conditioning2d の概要図．黒の部分は全ての値が 0 で構成されている行列・ベクトルで，白の部分は全ての値が 1 で構成されている行列・ベクトルである．この図は bass 特徴について学習する場合の条件付けの例．

時間方向に平均をとったのち，チャンネル方向と周波数方向を平坦化して全結合層で整形して 640 次元ベクトルになる．この 640 次元ベクトルに対しては先行研究 [11] と同様に図 3.3 上の条件付けを行う．具体的には 640 次元ベクトルの 128 次元ごとが各楽器音特徴になるようなマスクをかける．デコーダの入力に対しても出力特徴ベクトルと整合性の取れた条件付けをするために，エンコーダの出力の 640 チャンネルのうち 128 次元ごとが各楽器音特徴になるような条件付けを行う (図 3.3 下)．デコーダは着目する楽器音数分あり，本手法では 5 つ用意し，それぞれ特定の楽器音に対する条件付けがされたエンコーダ出力を入力する．

3.2.3　入力データ

ネットワークに個別楽器音特徴のみをネットワークに取り出させるために先行研究 [11] と同様，入力に擬似楽曲 (図 2.6) を用いる．また 2.3 節で述べた擬似楽曲の basic と additional，両方で学習を行う．擬似楽曲は短時間フーリエ変換により得られる振幅スペクトログラムに変換して，ネットワークに入力する．

また Disent では，トリプレット（アンカー，ポジティブ，ネガティブ）のデータセットを学習前に用意していたが，モデルが用意したデータに過度に適応してしまい，モデルの汎化性能が失われてしまうという可能性がある．そこで，提案手法では，学習時にミニバッチを構成する度に毎回トリプレットデータをランダムに生成する処理を導入する （Data Augmentation: DA）．

3.2.4 同時学習における損失関数

損失関数は，エンコーダの出力から算出される条件付けされた 640 次元ベクトルに対してはトリプレット損失を計算し，デコーダの出力に対しては入力楽曲を掛け合わせた推定分離音と教師分離音との L1 損失（Reconst 損失）を計算し，最後にその 2 つの損失を足し合わせたものを使用する．

まず特徴量ベクトルに対するトリプレット損失（式 2.1）について述べる．d は drums，b は bass，p は piano，g は guitar，r は residuals を表すとする．i 番目の入力データを $\mathbf{x}_i$ で表すとし，条件 $c \in \{d, b, p, g, r\}$ によって決定される条件付けマスクベクトルを $\boldsymbol{m}_c^{(1d)}$ で表すとする．またネットワークの U-Net エンコーダ部分を関数 $f_{\mathrm{e}}(\cdot)$，エンコーダ出力を特徴量ベクトルに変換するまでのネットワークを関数 $g(\cdot)$ で表すとすると，トリプレット損失（式 2.1）における特徴量ベクトル同士の距離を計算する距離関数 $d(\cdot)$ は以下で定式化できる．

$$d(\mathbf{x}_i, \mathbf{x}_j; c) = \| g(f_{\mathrm{e}}(\mathbf{x}_i)) \odot \boldsymbol{m}_c^{(1d)} - g(f_{\mathrm{e}}(\mathbf{x}_j)) \odot \boldsymbol{m}_c^{(1d)} \|_2 \tag{3.2}$$

なお，距離計算には L2 ノルムを使用する．

次にデコーダから出力される分離音マスクに対する L1 損失（Reconst 損失）について述べる．a, p, n は，トリプレット損失おけるアンカー，ポジティブ，ネガティブを表すとする．$u \in \{a, p, n\}$ として，ネットワークの入力を $\mathbf{x}_i^{(u)}$ として，U-Net デコーダ部分を関数 $f_{\mathrm{d}}(\cdot)$ と表し，エンコーダ入力に対してかけ合わせる条件付けマスク行列を $\boldsymbol{m}_c^{(2d)}$

とする．この時，条件 u によって決定される，出力分離音 $f_{\mathrm{d}}(f_{\mathrm{e}}(\mathbf{x}_i^{(u)}) \odot \boldsymbol{m}_c^{(2d)}) \odot \mathbf{x}_i^{(u)}$ に対応する教師データである正解分離音を $\mathbf{y}_i^{(u)(c)}$ と表すと，音源分離ロスである L1 損失（Reconst 損失）は以下で与えられる．

$$\mathcal{L}_{\mathrm{reconst}}(\mathbf{x}_i^{(u)}, \mathbf{y}_i^{(u)(c)}) = \| f_{\mathrm{d}}(f_{\mathrm{e}}(\mathbf{x}_i^{(u)}) \odot \boldsymbol{m}_c^{(2d)}) \odot \mathbf{x}_i^{(u)} - \mathbf{y}_i^{(u)(c)} \|_1 \tag{3.3}$$

以上より最終的な損失関数は以下の式で表される．

$$\begin{aligned} \mathcal{L} = \sum_{c \in \{b', a'\}} & \mathcal{L}_{\mathrm{reconst}}(\mathbf{x}_i^{(a)}, \mathbf{y}_i^{(a)(c)}) + \mathcal{L}_{\mathrm{reconst}}(\mathbf{x}_i^{(p)}, \mathbf{y}_i^{(p)(c)}) + \mathcal{L}_{\mathrm{reconst}}(\mathbf{x}_i^{(n)}, \mathbf{y}_i^{(n)(c)}) \\ & + \mathcal{L}_{\mathrm{triplet}}(\mathbf{x}_i^{(a)}, \mathbf{x}_i^{(p)}, \mathbf{x}_i^{(n)}; c) \end{aligned} \tag{3.4}$$

ここで b'，a' は擬似楽曲における basic と additional の楽器を表しており，それぞれ $\{d, b, p, g, r\}$ の中のうちの 1 つを表す．

3.3　提案手法 2：Serial-FT

Serial に対して，音源分離モデルを学習可能にし，音源分離モデルに対する事前学習と End-to-End な fine-tuning （FT）を導入したモデルを提案する．

3.3.1　ネットワーク構造

提案手法のネットワークの概要が図 3.4 である．Serial では図 3.4 の特徴量抽出器のみ学習を行い，音源分離モデルである図 3.4 の U-Net は spleeter [14] の学習済みモデルを使用しているため，学習を行わない．そこで提案手法ではこの音源分離モデルを学習可能にする．具体的には，まず音源分離モデルを事前学習し，次に特徴量抽出器のみを学習し，最後に End-to-End な fine-tuning （FT）を行うことで性能向上を目指す．

音源分離モデルである U-Net 部分のネットワーク構造は spleeter [14] や Jansson らの研究 [18] と同様である．特徴量抽出器のネットワーク構造は，先行研究 [10] では 4 層

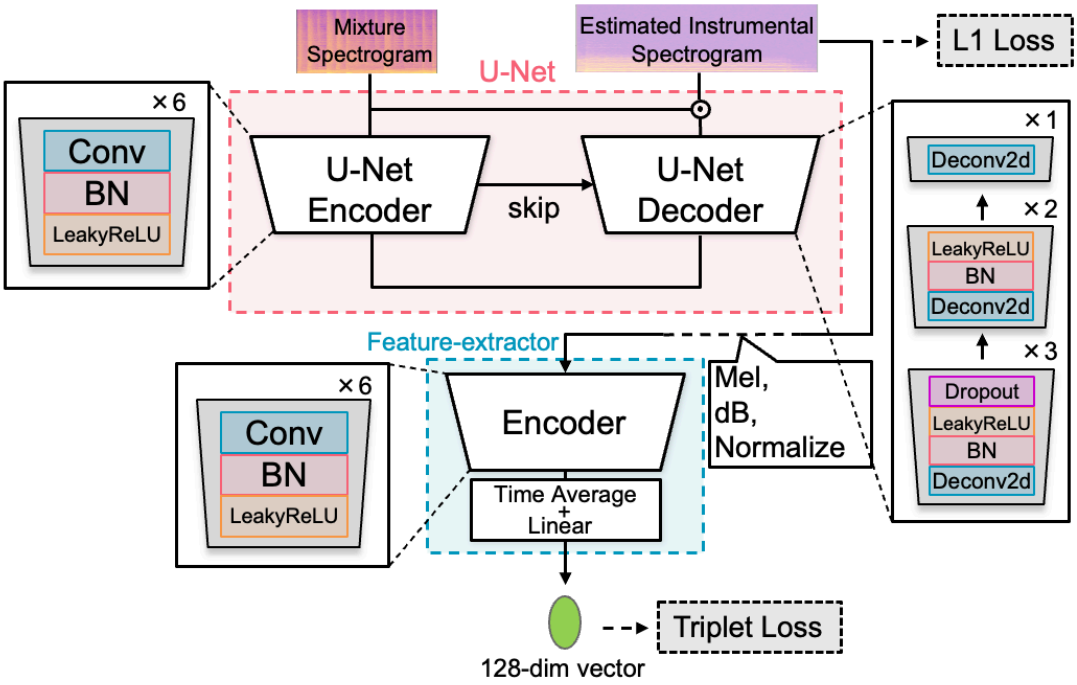


図 3.4: 提案手法 2：Serial-FT の概要図

の畳み込みを用いていたのに対し，提案手法では U-Net エンコーダ部分と同じ構造である 6 層の畳み込みを用いる．

3.3.2 音源分離モデルの学習

学習の手順としては，まず初めに音源分離モデルである U-Net の学習を事前学習として行う．先行研究 [10] では音源分離モデルには spleeter [14] の学習済みモデルを使用していたため，音源分離モデルは生音（Acoustic）の楽曲で学習されていた．これに対し，本論文では，特徴量抽出器の学習に用いるデータセット（slakh [13]，MIDI 音源）と同じものを用いて音源分離モデルを事前学習する．そこで MIDI 音源である slakh で音源分離モデルを事前学習させる．U-Net 部分の学習には spleeter [14] や Jansson らの研究 [18] と同様のネットワーク構造および学習手法を用いる．具体的には，U-Net を

関数 $f(\cdot)$ で表すと，i 番目の U-Net の入力 $\mathbf{x}_i$ と出力 $f(\mathbf{x}_i)$ をかけあわせたものが対象楽器音源になるように教師データである clean 楽器音 $\mathbf{y}_i$ との L1 損失を計算して学習を行う．損失関数を定式化すると以下のようになる．

$$\mathcal{L}_{\text{unet}}(\mathbf{x}_i, \mathbf{y}_i) = \|\mathbf{x}_i \odot f(\mathbf{x}_i) \ - \mathbf{y}_i\|_1 \tag{3.5}$$

3.3.3　特徴量抽出器の学習

U-Net を学習後，特徴量抽出器の学習を行う．特徴量抽出器の入力は学習した U-Net の出力と入力をかけ合わせた推定分離音であり，学習の際は U-Net 部分のパラメータは固定する．学習は，Serial と同様に，トリプレット損失（式 2.1）を用いて行う．特徴量抽出器を関数 $g(\cdot)$ で表し，サンプル間の距離を計算する距離関数を $d(\cdot)$ とすると，トリプレット損失中における距離関数 $d(\cdot)$ は以下で定式化できる．

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|g(\mathbf{x}_i \odot f(\mathbf{x}_i)) - g(\mathbf{x}_j \odot f(\mathbf{x}_j))\|_2 \tag{3.6}$$

なお，距離関数 $d(\cdot)$ には L2 ノルムを使用する．

3.3.4　fine-tuning

音源分離モデルと特徴量抽出器を学習した後，二つのパラメータどちらも固定せず End-to-End な fine-tuning （FT）を行う．具体的には音源分離モデルを学習する際に使用した L1 損失 (式 3.5) と特徴量抽出器を学習する際に使ったトリプレット損失 (式 2.1) を足したものを損失関数として使い学習をする．トリプレット損失におけるアンカー，ポジティブ，ネガティブの i 番目のデータを，$\mathbf{x}_i^{(a)}$，$\mathbf{x}_i^{(p)}$，$\mathbf{x}_i^{(n)}$ と定義し，それぞれと対応する教師 clean 楽器音を $\mathbf{y}_i^{(a)}$，$\mathbf{y}_i^{(p)}$，$\mathbf{y}_i^{(n)}$ と定義すると，損失関数は以下のよ

うに定式化できる．

$$\begin{aligned}\mathcal{L} =& \mathcal{L}_{\text{unet}}(\mathbf{x}_i^{(a)}, \mathbf{y}_i^{(a)}) + \mathcal{L}_{\text{unet}}(\mathbf{x}_i^{(p)}, \mathbf{y}_i^{(p)}) + \mathcal{L}_{\text{unet}}(\mathbf{x}_i^{(n)}, \mathbf{y}_i^{(n)}) \\ &+ \mathcal{L}_{\text{triplet}}(\mathbf{x}_i^{(a)}, \mathbf{x}_i^{(p)}, \mathbf{x}_i^{(n)})\end{aligned} \tag{3.7}$$

3.4 まとめ

第 3 章では，音源分離を利用した楽曲特徴量抽出モデルについて，Disent と Serial 双方について提案した．まず，3.2 節では，Disent に対して，Reconst を同時学習させる手法を提案した．また使用するトリプレットデータに対する DA についても述べた．さらに，3.3 節では，Serial に対して End-to-End な FT を提案した．また，音源分離モデルに対する事前学習手法や，特徴量抽出器に対する学習手法についても述べた．

第4章　実験的評価

4.1　はじめに

本章では，提案手法 1 の同時学習が個別楽器音特徴の分離表現学習に有効であること，提案手法 2 の音源分離モデルに対する事前学習と End-to-End な fine-tuning が有効であることを明らかにするため，slakh [13] のデータセットを用いた実験的評価を行う．

4.2　実験条件

4.2.1　データセット

使用するデータセットである slakh [13] は色々な楽器音から構成されている全 2100 曲の MIDI 楽曲データセットである．このデータセットには楽曲音源だけでなく，その楽曲音源を構成している各個別楽器音データも含まれている．そのため提案手法のように各個別楽器音データを組み合わせて擬似楽曲を作成することも可能である．さらに各楽曲には drums，bass，piano，guitar 音が必ず含まれており，先行研究 [10][11] および提案手法では楽曲中のこれらの楽器音以外の音を residuals として扱う．また，slakh の 2100 曲中には，同一の MIDI ファイルから生成された曲が複数組含まれている．本実験では，2100 曲から同一の MIDI ファイルから生成された曲を除いた曲を，学習用に 1200 曲，検証用に 270 曲，評価用に 136 曲用意し，使用する．

4.2.2　提案手法 1 に対する実験

提案手法 1 に対する実験について述べる．学習で用いるデータセットの楽曲は無音区間を排除しオーバーラップさせながら指定の時間長のセグメントに分割したものを用いる．分割後，短時間フーリエ変換を行い絶対値をとって振幅成分のみを取り出して使用する．学習の終了には設定した最大エポック数に達するか，検証用データにおける擬似楽曲に対する k 近傍法の値 (セクション 4.3 で後述) が 100 エポック最高値を更新しなかったら終了とする．提案手法 1 のデータセットに対する実験条件を表 4.1 に，モデルに対する実験条件を表 4.2 に示す．

4.2.3　提案手法 2 に対する実験

提案手法 2 に対する実験について述べる．学習に用いるデータセットの楽曲は提案手法 1 に対する実験と同様である．提案手法 2 の U-Net の出力に入力楽曲スペクトログラムをかけ合わせたものは，入力楽曲中の楽器分離スペクトログラムとなり，それを特徴量抽出器に入力することになるが，その前にメルフィルタバンクをスペクトログラムにかけメルスペクトログラムに変換し，対数をとってデシベル化，最後に標準化したのち入力する．また提案手法 1 と同じく，提案手法 2 でも擬似楽曲をランダムに生成し続け，エポック間でも異なる擬似楽曲を使用する．提案手法 2 のデータセットに対する実験条件を表 4.3 に，モデルに対する実験条件を表 4.4 に示す．

4.3　評価方法

4.3.1　テスト楽曲に対する特徴表現の精度評価

提案手法は先行研究 [10][11] と同じく，同じ楽曲から分割されたセグメントは類似しているという仮定に基づいて学習を行う．そこで，学習した特徴表現を評価するため

に，特徴表現の類似度を用いて推論した楽曲 ID の正解率を用いる．具体的には，k 近傍法を用いて，テストセグメントの楽曲 ID を推論する．推論するセグメント以外のテストセグメントの楽曲 ID は既知であると仮定し，全てのテストセグメントを学習した特徴表現空間に埋め込み，上位 5 つの最近傍テストセグメントの ID を用いた多数決により，各テストセグメントの楽曲 ID を予測する．評価は各楽器音に対して行うため，提案手法 1 のような特徴次元の一部が個別楽器音特徴を表しているような場合は対象次元以外の次元を 0 にマスクしてから k 近傍法を適用する．評価を行う際はテストデータセット全体を用いて正解率を算出する．

4.3.2 テスト用擬似楽曲に対する特徴表現の精度評価

提案手法は個別楽器音を特徴表現に分離して出力することを目指している．しかし 4.3.1 節で述べた，そのままの楽曲を用いた評価では，同じ楽曲から分割されたセグメントは類似しているという仮定のもとでは，全ての楽器音特徴量ベクトルは同じラベルが付与されることになる．そのため，例えば，drums 次元に guitar 特徴量や混合音特徴量が反映されていたとしても正しく推論されてしまう．そこで，出力された特徴量の楽器音特徴分離精度を調べるために，擬似楽曲を作成し，擬似楽曲に対して 4.3.1 節で行ったラベル推定を行う．作成するテスト用擬似楽曲の概要が図 4.1 である．k 近傍法の正解ラベルには対象楽器音のラベル (例の場合は drums のラベル) を使用する．

また，同じ構成の擬似楽曲のセグメントを含んだままテストを行ってしまうと，その同じ構成の擬似楽曲セグメントによるラベル推定のみ，そのままの楽曲を用いた場合の評価と同じく，他の楽器音特徴が反映されていても，正しく推論できてしまう．そこで推論時は同じ構成の擬似楽曲を含む他のセグメントは排除してテストを行う．またテストに用いる擬似楽曲は，対象楽器音ラベル 10 曲分で，対象楽器音 1 曲に対して対象以外の楽器音ラベルを 5 曲用意することで得られる計 50 曲を用いる．

	drums	bass	piano	guitar	residuals
Pseudo X1	Track01	Track02	Track02	Track02	Track02
Pseudo Y1	Track01	Track03	Track03	Track03	Track03
Pseudo Z1	Track01	Track04	Track04	Track04	Track04
⋮					
Pseudo X5	Track05	Track06	Track06	Track06	Track06
Pseudo Y5	Track05	Track07	Track07	Track07	Track07
⋮					

図 4.1: テスト用擬似楽曲の概要図．この図は drums を対象にテストを行う場合の擬似楽曲である．同じ構成の擬似楽曲は複数用意する．

4.3.3　主観評価

提案手法の出力した特徴量から得た類似度が人の知覚的な類似度基準を表現しているかどうかを調査するために主観評価を行う．以下に，piano 音に対する評価を例にその手順を説明する. テスト楽曲から 4 つの楽曲 X をランダムに選び，$X_i(i = 1, 2, 3, 4)$ を構成する piano 音から 10 秒間の無音でない部分を切り取って x_i とする. そして X_i とは異なる楽曲から，楽曲 A_i, B_i, C_i をランダムに選び，同様に各サンプル音から 10 秒間を切り取って，それぞれ a_i, b_i, c_i とする. また，楽曲 X_i の x_i とは異なる時間から抽出された 10 秒間のサンプルを y_i とする.4 つのサンプル x_i 各々に対し，それぞれ 2 つの評価セット set1$\{x_i, a_i, b_i\}$ と set2$\{x_i, y_i, c_i\}$ を 2 つずつ，計 $4 \times 2 \times 2 = 16$ セット作成する. このサンプルセットを他の楽器 (drums，bass，guitar) についても同様に作成し，こうして作成した 64 セットを被験者にランダムに聞かせ，a, b のどちらが x に似ているか，y, c のどちらが x に似ているかを回答させる．piano に対する評価の場合は，piano 音サンプルを聞いた際に，piano 部分空間において x を含む楽曲との距離が小さい方のサンプルを選択すれば正解とし，他の楽器においても同様の方法で評価し，正解率を算出する.

4.4 結果

4.4.1 提案手法1に対する評価結果

提案手法1に対する実験評価の結果について述べる．通常テスト楽曲に対する楽曲推定の評価結果を表4.5に，テスト用擬似楽曲に対する楽曲推定の評価結果を表4.6に，主観評価結果を表4.7に示す．

表4.6より，テスト用擬似楽曲に対する評価において，Disentと Reconstの同時学習に加え，DAを行うことで，先行研究[11]を大きく上回る性能を出すことができていることが確認できる．特に，DAの有無がテスト用擬似楽曲に対する評価値の向上に大きく貢献していることから，DAの各楽器音特徴量分離における有効性を確認できる．また，inputが melの場合のデータは先行研究[11]と同じく入力にメル対数スペクトログラムを用いた場合のモデルの値である．inputが melの場合のデータを比較してもReconstを同時学習させたモデルの方が評価値が高いことから，Reconstの同時学習が有効であることがわかる．

しかし，表4.5より，通常テスト楽曲に対する評価値は bass，piano，guitarでは依然先行研究[11]の値が最も高い．しかしながら，表4.6における Disent[11]のテスト用擬似楽曲に対する評価値より，Disent[11]の楽曲特徴は厳密に楽器音特徴をそれぞれの部分空間に反映できていないことから，先行研究[11]の通常テスト楽曲に対する評価値は様々な他の特徴量が加味された結果であり，楽器音特徴に対する評価値ではないと考えられる．提案手法はテスト用擬似楽曲に対する評価値を高く保ちながら，通常テスト楽曲に対する評価値も先行研究と同水準の値を出すことができている．

また，表4.7より，主観評価結果の値に関しては，set2に対しては提案手法が先行研究を上回る値が出ていることから，同一の楽曲のセグメント同士は類似しているという仮定が正しいこと，そして提案手法が同一の楽曲のセグメントから出力される特徴量同士を類似させることができていることがわかる．しかし，set1に対しては piano，

guitar の値に向上が見られているが，drums，bass においては値が下がる結果となっている．特に drums において，Disent[11] では約 70%の正解率が出ているが，提案手法では約 28%の値となっていて，著しい低下が見られることがわかる．このことから提案手法では，特に drums においては人の知覚的な類似性とは異なる類似性に基づいた特徴に着目している可能性が示唆される．一方で，主観評価に用いた楽曲数が少ないことから，選ばれた楽曲依存の結果である可能性も考えられるため，主観評価に対しては更なる検証が必要であるとも考えられる．

以上より提案手法 1 が楽器音特徴を部分空間に分離して出力でき，かつ類似した楽器音に対して類似した特徴量を出力できる点で Disent と DA，再構成の同時学習が最も優れていると言うことができる．

4.4.2　提案手法 2 に対する評価結果

提案手法 2 対する実験的評価の結果について述べる．通常テスト楽曲に対する楽曲推定の評価結果を表 4.8 に，テスト用擬似楽曲に対する楽曲推定の評価結果を表 4.9 に，主観評価結果を表 4.10 に示す．

表 4.8 より，通常テスト楽曲に対する評価結果では，Serial において，音源分離モデルに対する事前学習と，FT を適用した手法が bass 以外で最も値の高い結果となっている．さらに，事前学習を行なったモデルである Serial（MIDI）と spleeter を使用している Serial（Acoustic）を比較した時，drums と piano，中でも piano で性能向上が見られたことから，音源分離の事前学習が特徴量計算において有効であることが確認できる．また表 4.8，4.9 より，通常テスト楽曲に対する評価，テスト用擬似楽曲に対する評価ともに，FT を行わない手法と比較して FT を行った手法の評価値の方が全楽器音において高い値となっていることから，FT がどの楽器の特徴量計算においても有効であることが分かる．ただ，表 4.8 において，特徴量抽出器の入力に clean な楽器音を用いた場合の結果と提案手法を比較すると，提案手法の値は clean な音源を用いた場

合の値にはまだ届いていない．このことから，音源分離を行ったことによる楽器音の音質劣化，欠損等が特徴量抽出器の性能を下げる要因になっていることが確認できる．

また，表 4.10 より，提案手法に対する主観評価結果において，set2 における評価値は全て 80%以上であり，同一曲内の異なる時間区分に対して互いに類似した特徴量を出力することができていると言える．しかし set1 においては特に drums と bass において約 50%と低い値となっている．このことから Disent に対する提案手法と同じく，Serial に対する提案手法では，drums や bass において人の知覚的な類似性とは異なる類似性に基づいた特徴に着目している可能性が示唆される．

さらに先行研究と提案手法の音源分離性能を比較した．評価には SDR（Signal to Distortion Ratio）を用いる．この評価指標は分離音の品質を表す値であり，値が高いほど性能が良いことを表す．音源分離性能の評価結果を表 4.11 に示す．表 4.11 より，提案手法で用いた，slakh [13] を用いて事前学習させた音源分離モデルの方が，spleeter[14] より全楽器音において分離精度が大幅に高いことが分かる．表 4.8 より，特に piano における通常テスト楽曲に対する評価値が向上していることから，楽器の種類によっては分離精度の向上が特徴量計算精度の向上をもたらすことが分かる．また FT を導入したモデルと導入していないモデルを比較しても分離精度に大きな違いが見られていないことから，FT は音源分離モデルの性能にはあまり影響を与えないことが分かる．このことから FT は音源分離モデルの性能より特徴量抽出器の性能に改善をもたらすと言える．

以上より，Serial において音源分離の事前学習と FT が同一楽曲内の楽器音に対して類似した特徴量を出力できるという点で最も優れたモデルであると言える．

4.5　まとめ

第 4 章では, Disent と Serial の両手法に対する提案手法に対して実験的評価を行った. まず，4.2 節では，各提案手法に対する実験条件を確認した．そして，4.3 節では，提案手法の有効性を示すための評価指標とその評価方法を述べた．最後に，4.4 節では，提案手法に対する評価結果を示した．評価結果より，Disent に対しては，DA と Reconst の同時学習によって性能が改善されること，そして Serial に対する FT と音源分離モデルの事前学習がそれぞれ有効であることが示された.

表 4.1: 実験条件：提案手法 1 データセット

	Train	Dev	Test
The number of songs	1200	270	136
Sample rate	44100	44100	44100
Slice length	3s	10s	10s
Over lap rate	50%	0%	0%
STFT frame size	2048	2048	2048
STFT offset	512	512	512
Conversion to mel-spec	No	No	No
Conversion to dB	No	No	No
Normalizing input data	No	No	No

表 4.2: 実験条件：提案手法 1 モデル

Optimizer	Adam[19]
Learning rate	0.0001
Max epoch	400
Batch size	30
The number of triplets	20000
Convolution: Layer1's channel output	16
Convolution: Layer2's channel output	32
Convolution: Layer3's channel output	64
Convolution: Layer4's channel output	128
Convolution: Layer5's channel output	256
Convolution: Layer6's channel output	640
Convolution: Kernel Size	5
Convolution: Stride	2
Convolution: Padding	2
Leaky ReLU's negative slope	0.2
Drop out rate	50%
Triplet loss's margin	0.2

表 4.3: 実験条件：提案手法 2 データセット

	Train	Dev	Test
The number of songs	1200	270	136
Sample rate	44100	44100	44100
Slice length	3s	10s	10s
Over lap rate	50%	0%	0%
STFT: frame size	2048	2048	2048
STFT: offset	512	512	512
U-Net: Conversion to mel-spec	No	No	No
U-Net: Conversion to dB	No	No	No
U-Net: Normalizing input data	No	No	No
Feature-extractor: Conversion to mel-spec	Yes	Yes	Yes
Feature-extractor: Mel-Frequency	259	259	259
Feature-extractor: Conversion to dB	Yes	Yes	Yes
Feature-extractor: Normalizing input data	Yes	Yes	Yes

表 4.4: 実験条件：提案手法 2 モデル

Optimizer	Adam[19]
Learning rate （Pre-train）	0.001
Learning rate （Triplet learning）	0.001
Learning rate （fine-tuning）	0.00001
Batch size	64
The number of triplets	20000
U-Net, Feature-extractor: Layer1's channel out	16
U-Net, Feature-extractor: Layer2's channel out	32
U-Net, Feature-extractor: Layer3's channel out	64
U-Net, Feature-extractor: Layer4's channel out	128
U-Net, Feature-extractor: Layer5's channel out	256
U-Net, Feature-extractor: Layer6's channel out	640
U-Net, Feature-extractor: Kernel size	5
U-Net, Feature-extractor: Stride	2
U-Net, Feature-extractor: Padding	2
U-Net, Feature-extractor: Leaky ReLU's negative slope	0.2
U-Net, Feature-extractor: Drop out rate	50%
Triplet loss's margin	0.2

表 4.5: 提案手法 1：通常テスト楽曲に対する楽曲推定の評価結果 (%)．Disent[11] の評価値は先行研究 [11] から引用.

Method	input	drums	bass	piano	guitar	residuals
Disent[11]	mel	89.69	**84.45**	**85.70**	**86.27**	84.86
Disent+DA	mel	89.33	71.09	79.74	81.75	85.67
Disent+DA+Reconst	mel	89.39	79.46	83.90	85.92	86.69
Disent+DA+Reconst	spec	**91.14**	81.30	84.76	85.17	**88.84**

表 4.6: 提案手法 1：テスト用擬似楽曲に対する楽曲推定の評価結果 (%)．Disent[11] の評価値は先行研究 [11] から引用.

Method	input	drums	bass	piano	guitar	residuals
Disent[11]	mel	85.5	37.1	31.3	44.7	74.7
Disent+DA	mel	97.93	68.22	69.22	63.24	89.99
Disent+DA+Reconst	mel	97.16	70.99	78.59	75.00	**95.59**
Disent+DA+Reconst	spec	**98.25**	**77.74**	**79.20**	**82.47**	94.62

表 4.7: 提案手法 1：主観評価結果の正解率と 95%信頼区間 (%)．Disent[11] の評価値は先行研究 [11] から引用.

Sets	Methods	drums	bass	piano	guitar
set1	Disent[11]	**70.8 ± 8.2**	**53.3 ± 9.0**	44.2 ± 8.9	84.2 ± 6.6
	Disent+DA+Reconst	28.33 ± 8.10	44.17 ± 8.92	**70.00 ± 8.23**	**85.00 ± 6.42**
set2	Disent[11]	82.5 ± 6.8	**98.3 ± 2.3**	74.2 ± 7.9	**80.8 ± 7.1**
	Disent+DA+Reconst	**91.67 ± 4.57**	**98.33 ± 2.30**	**97.50 ± 2.81**	**80.83 ± 7.11**
total	Disent[11]	**76.7 ± 5.4**	**75.8 ± 5.4**	59.2 ± 6.2	82.5 ± 4.8
	Disent+DA+Reconst	60.00 ± 6.21	71.25 ± 5.74	**83.75 ± 4.68**	**82.92 ± 4.77**

表 4.8: 提案手法 2：通常テスト楽曲に対する楽曲推定の評価結果 (%)．Clean[10] は特徴量抽出器の入力に分離楽器音ではなく clean な楽器音を用いた場合の結果である．Clean[10] と Serial（Acoustic）[10] の評価値は先行研究 [10] から引用．

Method	drums	bass	piano	guitar
Clean[10]	98.04	94.60	98.14	96.35
Serial（Acoustic）[10]	88.91	**63.87**	50.34	-
Serial（MIDI）	89.66	58.81	74.34	72.69
Serial（MIDI）+FT	**90.53**	62.73	**78.83**	**76.65**

表 4.9: 提案手法 2：テスト用擬似楽曲に対する楽曲推定の評価結果 (%)．先行研究 [10] では同様の評価が行われていないため，Serial（Acoustic）[10] の評価値は空欄．

Method	drums	bass	piano	guitar
Serial（Acoustic）[10]	-	-	-	-
Serial（MIDI）	**99.73**	91.20	89.87	91.73
Serial（MIDI）+FT	**99.73**	**93.07**	**92.67**	**93.47**

表 4.10: 提案手法 2：主観評価結果の正解率と 95%信頼区間 (%)．

Sets	Methods	drums	bass	piano	guitar
set1	Serial（MIDI）+FT	55.00 ± 8.94	52.50 ± 8.97	60.83 ± 8.77	79.17 ± 7.30
set2	Serial（MIDI）+FT	91.67 ± 4.97	85.83 ± 6.27	97.50 ± 2.81	80.00 ± 7.19
total	Serial（MIDI）+FT	73.33 ± 5.61	69.17 ± 5.85	79.17 ± 5.15	79.58 ± 5.11

表 4.11: 提案手法 2：音源分離モデルの slakh [13] に対する分離精度の評価値．評価には SDR（Signal to Distortion Ratio）を使用．Serial（Acoustic）[10] の結果は先行研究 [10] から引用．なお提案手法の SDR 算出には museval[20] を用いた．

	SDR			
Method	drums	bass	piano	guitar
Serial（Acoustic）[10]	-13.7	-15.5	14.7	-
Serial（MIDI）	**15.50**	10.54	**7.81**	6.94
Serial（MIDI）+FT	15.44	**10.88**	7.62	**7.08**

第5章　結論

5.1　本論文のまとめ

本論文では，各個別楽器音特徴に基づく楽曲間類似度を計算する 2 つの先行研究，Serial，Disent における問題の解消に向けて，Disent に対しては，DA と Reconst を同時学習する手法を，Serial に対しては音源分離モデルの事前学習と End-to-End な FT を行う手法を提案した．

第 2 章では，まず，各個別楽器音特徴に基づく楽曲間類似度を計算する手法における，音源分離モデルと特徴量抽出器を直列（Serial）に接続する手法について述べ，また，同じく各個別楽器音特徴量に基づく楽曲間類似度を出力する手法における，単一の特徴量抽出器に楽曲音源そのものを入力して，各楽器音の特徴量を部分空間に分離して（Disentanglement: Disent）出力する手法について述べた．

第 3 章では，Disent に対してはデータ拡張（Data Augmentation: DA）と楽器音の再構成（Reconstruction: Reconst）の同時学習，Serial に対しては音源分離部分への事前学習と End-to-End な fine-tuning（FT）を提案し，各提案手法の詳細について述べた．

第 4 章では，各提案手法に対する実験的評価の方法と評価指標，そしてその評価結果について述べ，Disent に対する DA と Reconst の同時学習がそれぞれ有効であること，そして Serial に対して，音源分離モデルの事前学習と FT が有効であることについて述べた．

5.2　今後の課題

第 4 章で述べたように，提案手法の主観評価より，提案システムから出力された特徴量から得られる類似度と人の聴覚的な類似度が一致していない点が問題点として挙げられる．提案手法・先行研究ともに特徴抽出器には畳み込み層と，プーリング方法に時間平均を用いているが，音楽的な類似度を算出する上で，畳み込み層で得られる局所的特徴だけではなく，時間方向の系列情報から得られる特徴が必要不可欠であると考えられる．特に drums のような音楽にリズム情報を付加する楽器音や，bass のような低周波の楽器音には，局所的な特徴より時間方向の系列特徴が，音楽的類似度を計算する上で，重要な要素である可能性が高い．しかし提案手法・先行研究ともに長期的な時系列特徴を計算することが難しく，この時系列情報を失った結果，提案システムの出力特徴量から得られる類似度と人の聴覚的な類似度が乖離してしまっていると考えられる．上記の課題を踏まえ，今後は時間方向の系列特徴を捉えることのできる時系列アプローチと，時間平均を用いない時間方向のプーリング方法を検討していきたい．時系列アプローチと時間平均を用いない時間方向のプーリング方法によって，人の聴覚的な類似度と近い類似度を表現できる特徴量を計算できるシステムの開発が期待される．

また，本実験で用いた slakh [13] のようなステム音源は，一般的な楽曲では公開されていないことが多く，学習に使用できるデータ量にも限りがあると考えられる．加えて，slakh [13] は MIDI 音源であることに対して，一般的にストリーミングで配信されているような曲は生音楽曲であり，用いている楽曲の生成法に違いがある．生音楽曲は，実際に人間が演奏しているゆえ，MIDI 楽曲よりもランダム性のあるデータであると考えられるため，生音を扱った実験を行った場合，MIDI 楽曲を扱った本研究の実験結果より劣った結果が出てしまうと考えられる．実用的な楽曲検索・推薦システムを構築するためには，より実用的環境に近い，より制限された条件下での実験で高い精度を達成しなければいけないと考えられるため，更なる性能改善と実環境に適応でき

るアーキテクチャの構築が求められると考えられる.

謝辞

本研究を進めるにあたり，終始熱心な御指導，助言をして頂きました名古屋大学大学院情報学研究科 戸田智基 教授に心より感謝の意を表します.

また本研究を進めるにあたり，さまざまなアドバイスをして頂き，また研究に取り組む姿勢を御指導頂いた名古屋大学大学院情報学研究科戸田研究室 橋爪優香 修士に心より感謝の意を表します.

また，基礎学習から研究内容に至るまで多大な協力と助言を頂きました名古屋大学大学院情報学研究科武田研究室 武田一哉 教授, 藤井慶輔 准教授に心から感謝致します.

最後に，日ごろから共に研究を行い，多くの助言，協力を頂いた武田研究室・戸田研究室の先輩諸氏，ともに励ましあい切磋琢磨してきた学生諸氏に感謝申し上げます.

参考文献

[1] IFPI. Global music report 2023, 2023. https://www.ifpi.org/wp-content/uploads/2020/03/Global_Music_Report_2023_State_of_the_Industry.pdf.

[2] Spotify. Culture next volume 5, 2023. https://go.pardot.com/l/52662/2023-10-23/ljk7xt/52662/169805013966KGzgtB/Spotify_2023_Culture_Next_Report_JP_v3.pdf.

[3] Luminate. Top entertainment trends for 2023: What the data says. In *SXSW*, 2023. https://luminatedata.com/reports/sxsw-top-entertainment-trends-for-2023/.

[4] S. Dixon Y. Song and M. Pearce. A survey of music recommendation systems and future perspectives. In *International Symposium on Computer Music Modeling and Retrieval*, pp. 395–410, 2012.

[5] Y. V. S. Murthy and S. G. Koolagudi. Content-based music information retrieval (cb-mir) and its applications toward the music industry: A review. *ACM Computing Surveys*, Vol. 51, No. 3, pp. 1–46, 2018.

[6] A. Elbir and N. Aydin. Music genre classification and music recommendation by using deep learning. *Electronics Letters*, Vol. 56, No. 12, pp. 627–629, 2020.

[7] M. Won, S. Oramas, O Nieto, F. Gouyon, and X. Serra. Multimodal metric learning for tag-based music retrieval. In *International Conference on Acoustics, Speech and Signal Processing*, pp. 591–595, 2021.

[8] Y-J. Luo, K. Agres, and D. Herremans. Learning disentangled representations of timbre and pitch for musical instrument sounds using gaussian mixture variational autoencoders. *International Society for Music Information Retrieval*, pp. 746–753, 2019.

[9] Y-N. Hung, Y-A. Chen, and Y-H. Yang. Learning disentangled representations for timber and pitch in music audio. In *arXiv e-prints: 1811.03271*, 2018. 10 pages.

[10] Y. Hashizume, L. Li, and T. Toda. Music similarity calculation of individual instrumental sounds using metric learning. *Proceedings of 2022 APSIPA Annual Summit and Conference*, pp. 33–38, 2022.

[11] 橋爪優果, 李莉, 宮下敦志, 戸田智基. 個別楽器音に基づいた楽曲間類似度のための分離表現学習. 情報処理研報, Vol. 2023-MUS-137, No. 9, pp. 1–7, 2023.

[12] E. Hoffer and N. Ailon. Deep metric learning using triplet network. In *Similarity-Based Pattern Recognition*, pp. 84–92, 2015.

[13] E. Manilow, G. Wichern, P. Seetharaman, and J. L. Roux. Cutting music source separation some slakh: A dataset to study the impact of training data quality and quantity. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, pp. 45–49, 2019.

[14] R. Hennequin, A. Khlif, F. Voituret, and M. Moussallam. Spleeter: A fast and state-of-the art music source separation tool with pre-trained models. *The Journal of Open Source Software*, Vol. 5, No. 50, p. 2154, 2019. Deezer Research.

[15] A. Veit, S. Belongie, and T. Karaletsos. Conditional similarity networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1781–1789, 2017.

[16] J. Lee, N. J. Bryan, J. Salamon, Z. Jin, and J. Nam. Disentangled multidimensional metric learning for music similarity. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*, pp. 6–10, 2020.

[17] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention*, pp. 234–241, 2015.

[18] A. Jansson, E. J. Humphrey, N. Montecchio, R. M. Bittner, A. Kumar, and T. Weyde. Singing voice separation with deep u-net convolutional networks. In *International Society for Music Information Retrieval Conference*, pp. 23–27, 2017.

[19] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2014. 13 pages.

[20] F-R. Stöter, A. Liutkus, and N. Ito. The 2018 signal separation evaluation campaign. In *Latent Variable Analysis and Signal Separation*, pp. 293–305, 2018.